

SDCTF 2022 Crypto - Tasty Crypto Roll

deuterium — 2022-05-10

Canonical HTML: https://deut-erium.github.io/WriteUps/2022/sdctf/crypto/tasty_crypto_roll/2022-05-10-SDCTF-2022-Tasty-Crypto-Roll.html

Abstract

CTF writeup: SDCTF 2022's Tasty Crypto Roll seasons its key with a PID. Recover it, undo bit shuffling and let Z3 solve the ECB substitution constraints.

Tasty Crypto Roll

Description

CRYPTO - Hard

Tasty Crypto Roll

Bob, the genius intern at our company, invented AES-improved. It is based on AES but with layers after layers of proprietary encryption techniques on top of it.

The end result is an encryption scheme that achieves both confusion and diffusion. The more layers of crypto you add, the better the security, right?

Encrypter

[encrypt.py](#)

Encrypted file

[enc.bin](#)

Note

The intended solution requires very little brute force and runs under 5 seconds on our machine.

By k3v1n

Source

```
import os
import random
import secrets
import sys
from Crypto.Cipher import AES

ENCODING = 'utf-8'

def generate_key():
    return os.getpid(), secrets.token_bytes(16)

def to_binary(b: bytes):
    return ''.join(['{:08b}'.format(c) for c in b])
```

```

def from_binary(s: str):
    return bytes(int(s[i:i+8], 2) for i in range(0, len(s), 8))

def encrypt(key: bytes, message: bytes):
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.encrypt(message)

key1, key2 = generate_key()

print(f'Using Key:\n{key1}:{key2.hex()}')

def get_flag():
    flag = input('Enter the flag to encrypt: ')
    if not flag.startswith('sdctf{') or not flag.endswith('}') or not flag.isascii():
        print(f'{flag} is not a valid flag for this challenge')
        sys.exit(1)
    return flag

plaintext = get_flag()[6:-1]

data = plaintext.encode(ENCODING)

codes = list(''.join(chr(i) * 2 for i in range(0xb0, 0x1b0)))
random.seed(key1)
random.shuffle(codes)
sboxes = [''.join(codes[i*4:(i+1)*4]) for i in range(128)]

if len(set(sboxes)) < 128:
    print("Bad key, try again")
    sys.exit(1)

data = ''.join(sboxes[c] for c in data).encode(ENCODING)
data = encrypt(key2, to_binary(data).encode(ENCODING))

random.seed(key1)
key_final = bytes(random.randrange(256) for _ in range(16))

data_bits = list(to_binary(data))
random.shuffle(data_bits)
data = from_binary(''.join(data_bits))

ciphertext = encrypt(key_final, data)

print(f'Encrypted: {ciphertext.hex()}')
with open('enc.bin2', 'wb') as ef:
    ef.write(ciphertext)

```

Analysis

Here we can see mainly two parts

1. There are two keys
 - key1: pid of current process
 - key2: secure random key of 16 bytes

2. key1 is used as seed at a lot of places and is bruteforceable (2^{15}) key_final and sboxes are derived from key1, shuffling is done using key1

Steps to crack

1. decrypt using key_final
2. convert the intermediate ciphertext to_binary
3. de-shuffle the bits
4. generate from_binary intermediate ciphertext of the deshuffled bits
5. decrypt using key2???

How to find key1?

Assume you have the correct key1, reverse for the key, validate the results using some validator/logical assumption.

codes is a list of $2 * (0x1b0 - 0xb0) = 512$ characters, utf-8 encoding of which is 2-bytes each
sboxes will have 4char strings, which encode to 8 bytes each on utf-8 (i.e after substitution)
data is now $4 * 2 = 8$ times each byte of the original plaintext
data is converted to_binary before encryption hence each byte is converted to 8 b"0" or b"1" byte. Hence each character is substituted to some
 $8 * 8 = 64$ byte string before encryption.

Hence len of flag = $\text{len}(\text{ciphertext}) // 64 = 3520 // 64 = 55$ bytes

Assumption 1

Since length of flag is 55 characters, would it be reasonable to assume that there would be repetitions of characters. And since each flag character is substituted to fixed 64-byte strings before encryption which is a multiple of AES block size of 16, AES also acts like simple substitution of the flag but we do not know the mapping.

Hence if we reverse till step 4 above, we can simply check if there are any repeating 64-byte blocks, as incorrect shuffling of bits will result in each block to be distinct with almost 1 probability.

```
with open('enc.bin', 'rb') as f:
    ciphertext = f.read()

def to_binary(b: bytes):
    return ''.join(['{:08b}'.format(c) for c in b])

def from_binary(s: str):
    return bytes(int(s[i:i+8], 2) for i in range(0, len(s), 8))

def encrypt(key, message):
    return AES.new(key, AES.MODE_ECB).encrypt(message)

def decrypt(key: bytes, message: bytes):
    return AES.new(key, AES.MODE_ECB).decrypt(message)
```

```

def unshuffle(data_list, shuffle_order):
    res = [None]*len(data_list)
    for i,v in enumerate(shuffle_order):
        res[v] = data_list[i]
    return res

def key_final_dec(key1, ciphertext):
    random.seed(key1)
    key_final = bytes(random.randrange(256) for _ in range(16))

    data = decrypt(key_final, ciphertext)
    data_bits = list(to_binary(data))
    data_bits_order = list(range(len(data_bits)))
    random.shuffle(data_bits_order)
    data_bits_uns = unshuffle(data_bits, data_bits_order)
    data = from_binary(''.join(data_bits_uns))
    return data

```

Lets add a few validation too

```

def key_final_enc(key1, data):
    random.seed(key1)
    key_final = bytes(random.randrange(256) for _ in range(16))
    data_bits = list(to_binary(data))
    random.shuffle(data_bits)
    data = from_binary(''.join(data_bits))
    return encrypt(key_final, data)

def test_unshuffle():
    random_text = list(random.randbytes(16*1337))
    random_text_shuffled = random_text.copy()
    shuffle_order = list(range(len(random_text)))
    random.seed(1337)
    random.shuffle(random_text_shuffled)
    random.seed(1337)
    random.shuffle(shuffle_order)
    assert unshuffle(random_text_shuffled, shuffle_order) == random_text

def test_key_final_dec():
    random_text = random.randbytes(16*100)
    assert key_final_dec(1337, key_final_enc(1337, random_text)) == random_text

test_unshuffle()
test_key_final_dec()

```

Looks like all the decryption functions are correct, lets proceed with bruteforcing for key1

```

for key1 in tqdm(range(2**15), desc='solving for key1'):
    data = key_final_dec(key1, ciphertext)
    substitutions = Counter(data[i:i+64] for i in range(0, len(data), 64))
    if len(substitutions) != len(data)//64:
        print("pid =", key1)
        break

```

After waiting for an eternity, and exhausting the search space of possible pid's yet not getting any key1 got me confused. I checked my script locally for a test flag it seemed to work fine. There could only be one possibility

the flag contains 55 distinct characters

But how would I find key1 now?

Missed Catch

@Utaha#6878 pointed out, that since there are only 256 distinct values in codes each repeated twice, and each character encoded to some b"0" or b"1"

byte strings of length 16, It must be encrypted to the same block always.

Since the flag is $55 \times 4 = 220$ such 16-byte codes and each code is used twice for most of the characters, there will be repeating 16-byte blocks even with distinct flag characters.

Assumption 2

```
for key1 in tqdm(range(2**15), desc='solving for key1'):
    data = key_final_dec(key1, ciphertext)
    substitutions = Counter(data[i:i+16] for i in range(0, len(data), 16))
    if len(substitutions) != len(data) // 16:
        print("pid =", key1)
        break
```

pid = 83

And we found our key1!

And we can confirm that the flag is indeed 55 distinct characters.

Wait, if the flag is 55 distinct characters, how will we solve for the subs?

We have no statistical advantage and hence bye bye Mr quipquip

How do we find mapping for substitution?

Each sbx entry is composed of 4 2-byte strings, which can be one of 256 possible values. Moreover, their order is fixed, which is determined by key1.

If we try to solve for all valid mappings for AES(binary(sbx(char))) we will probably end up on the correct mapping and get our flag.

+-----+-----+-----+-----+				+-----+-----+-----+-----+															
flag0				flag1				flag55											
+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+							
sbx				sbx				sbx				sbx							
+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+							
c1		c2		c3		c4		c5		c6		c7		c8					
+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+							
AES				AES															
+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+				+-----+-----+-----+-----+							

```

|   |   +-----+
|   +--+       |
+-----+-----+-----+-----+
|E(c1) | E(c2) | E(c3) | E(c4)|
+-----+-----+-----+-----+

```

Enter Z3

We can assume our flag to be a list of BitVec of 7 bits each

And let the sboxes be a mapping from 7 bits to 64 bits each (16x4)

This can be achieved by assuming sbbox to be an array which is indexed by BitVec(7) and contains elements of BitVec(64)

And we assume AES to be some function from BitVec(16) to BitVec(128)

```

flag = [BitVec('flag_'+str(i),7) for i in range(len(data)//64)]
sboxmap = Array('sbox',BitVecSort(7), BitVecSort(64))
aes_encryption = Function('AES',BitVecSort(16), BitVecSort(128))

```

```

codes = list(''.join(chr(i) * 2 for i in range(0xb0, 0x1b0)))
random.seed(key1)
random.shuffle(codes)
# keeping sboxes utf encoded already
sboxes = [''.join(codes[i*4:(i+1)*4]).encode() for i in range(128)]

sbytes = b''.join(sboxes)
sboxints = list(map(lambda x:int.from_bytes(x,'big'),
                    set(sbytes[i:i+2] for i in range(0,len(sbytes),2))))
# integer values for 2-byte codes from sbox, will be explained shortly

sboxes = [int.from_bytes(i,'big') for i in sboxes]
data = key_final_dec(key1, ciphertext)
# converting intermediate decryption to 128 bit ints
data_int = []
for i in range(0,len(data),16):
    data_int.append(int.from_bytes(data[i:i+16],'big'))

```

```

# we know the sbox already
constraints = [sboxmap[i]==sboxes[i] for i in range(128)]
for i in range(len(data)//64):
    four_code = sboxmap[flag[i]]
    # splitting 64 bit quantity to 16 bit individual sbox codes
    four_code_parts = [Extract(16*i+15,16*i,four_code) for i in range(3,-1,-1)]
    # for each code, matching aes_encryption with the observed value
    for a,b in zip(data_int[4*i:4*i+4], four_code_parts):
        constraints.append(aes_encryption(b)==a)
    # last but not least, aes_encryption(i) is unique for each plaintext
    # how would z3 know? Distinct function encodes them appropriately to
    # be distinct
    constraints.append(Distinct([aes_encryption(i) for i in sboxints]))

solver = Solver()
solver.add(constraints)
for m in all_smt(solver, flag):
    # lets check for all satisfying flags (in case there are more than one

```

```
# possible mappings and we will rule out invalid ones in that scenario?
flag_bytes = bytes([m.eval(flag[i]).as_long() for i in range(len(flag))])
assert len(set(flag_bytes)) == len(Counter(data[i:i+64] for i in
range(0, len(data), 64)))
print(flag_bytes)
```

Flag

After running the script, we finally get our flag!

```
b'r0ll-uR~pWn.c6yPt0_wi7h,ECB:I5*b8d!KQvJmLxgX9DsaANMFSeU'
```

And it turns out to be the only satisfying assignment.

Turns out if there were repeated characters in the flag, we will get multiple possible satisfying values. So the admins have not been so cheeky afterall

Full script

Note that it takes a couple of seconds to find the z3 model

```
import random
from Crypto.Cipher import AES
from collections import Counter
from tqdm import tqdm
from z3 import *
import sys

def all_smt(s, initial_terms):
    def block_term(s, m, t):
        s.add(t != m.eval(t))

    def fix_term(s, m, t):
        s.add(t == m.eval(t))

    def all_smt_rec(terms):
        if sat == s.check():
            m = s.model()
            yield m
            for i in range(len(terms)):
                s.push()
                block_term(s, m, terms[i])
                for j in range(i):
                    fix_term(s, m, terms[j])
                yield from all_smt_rec(terms[i:])
            s.pop()
    yield from all_smt_rec(list(initial_terms))

with open('enc.bin', 'rb') as f:
    ciphertext = f.read()

def to_binary(b: bytes):
    return ''.join(['{:08b}'.format(c) for c in b])
```

```

def from_binary(s: str):
    return bytes(int(s[i:i + 8], 2) for i in range(0, len(s), 8))

def encrypt(key, message):
    return AES.new(key, AES.MODE_ECB).encrypt(message)

def decrypt(key: bytes, message: bytes):
    return AES.new(key, AES.MODE_ECB).decrypt(message)

def key_final_enc(key1, data):
    random.seed(key1)
    key_final = bytes(random.randrange(256) for _ in range(16))
    data_bits = list(to_binary(data))
    random.shuffle(data_bits)
    data = from_binary(''.join(data_bits))
    return encrypt(key_final, data)

def unshuffle(data_list, shuffle_order):
    res = [None] * len(data_list)
    for i, v in enumerate(shuffle_order):
        res[v] = data_list[i]
    return res

def test_unshuffle():
    random_text = list(random.randbytes(16 * 100))
    random_text_shuffled = random_text.copy()
    shuffle_order = list(range(len(random_text)))
    random.seed(10)
    random.shuffle(random_text_shuffled)
    random.seed(10)
    random.shuffle(shuffle_order)
    assert unshuffle(random_text_shuffled, shuffle_order) == random_text

test_unshuffle()

def key_final_dec(key1, ciphertext):
    random.seed(key1)
    key_final = bytes(random.randrange(256) for _ in range(16))

    data = decrypt(key_final, ciphertext)
    data_bits = list(to_binary(data))
    data_bits_order = list(range(len(data_bits)))
    random.shuffle(data_bits_order)
    data_bits_uns = unshuffle(data_bits, data_bits_order)
    data = from_binary(''.join(data_bits_uns))
    return data

def test_key_final_dec():
    random_text = random.randbytes(16 * 100)
    assert key_final_dec(10, key_final_enc(10, random_text)) == random_text

```

```

test_key_final_dec()

for key1 in tqdm(range(2**15), desc='solving for key1'):
    data = key_final_dec(key1, ciphertext)
    substitutions = Counter(data[i:i + 16] for i in range(0, len(data), 16))
    if len(substitutions) != len(data) // 16:
        print("pid =", key1)
        break

codes = list(''.join(chr(i) * 2 for i in range(0xb0, 0x1b0)))
random.seed(key1)
random.shuffle(codes)
sboxes = [''.join(codes[i * 4:(i + 1) * 4]).encode() for i in range(128)]
sbytes = b''.join(sboxes)
sboxints = list(map(lambda x: int.from_bytes(x, 'big'), set(
    sbytes[i:i + 2] for i in range(0, len(sbytes), 2))))
sboxes = [int.from_bytes(i, 'big') for i in sboxints]
data = key_final_dec(key1, ciphertext)
data_int = []
for i in range(0, len(data), 16):
    data_int.append(int.from_bytes(data[i:i + 16], 'big'))

flag = [BitVec('flag_' + str(i), 7) for i in range(len(data) // 64)]
sboxmap = Array('sbox', BitVecSort(7), BitVecSort(64))
aes_encryption = Function('AES', BitVecSort(16), BitVecSort(128))

constraints = [sboxmap[i] == sboxes[i] for i in range(128)]
for i in range(len(data) // 64):
    four_code = sboxmap[flag[i]]
    four_code_parts = [Extract(16 * i + 15, 16 * i, four_code)
                       for i in range(3, -1, -1)]
    for a, b in zip(data_int[4 * i:4 * i + 4], four_code_parts):
        constraints.append(aes_encryption(b) == a)
    constraints.append(Distinct([aes_encryption(i) for i in sboxints]))
solver = Solver()
solver.add(constraints)
# if solver.check() == sat:
# m = solver.model()
for m in all_smt(solver, flag):
    flag_bytes = bytes([m.eval(flag[i]).as_long() for i in range(len(flag))])
    assert len(set(flag_bytes)) == len(
        Counter(data[i:i + 64] for i in range(0, len(data), 64)))
    print(flag_bytes)
else:
    print("failed to solve")

```

Alternate Solution by teammate (Utaha#6878)

All due regards to him for solving the challenge while I was stuck over finding key1 XD

All parts will be almost same except the substitution solving part, which he did by manual bruteforcing i.e. recursively enumerating all mappings and backtracking on contradictions

```

mp = dict()
codes = sum([i, i] for i in range(256)), start=[]
# notice that the range is changed from [0xb0, 0x1b0) to [0, 256).
# It's just for relabeling.
random.seed(key1)
random.shuffle(codes)
sboxes = [codes[i*4:(i+1)*4] for i in range(128)]

def match(a, b):
    """
    equate two objects elementwise ignoring if the entry is -1
    """
    for x, y in zip(a, b):
        if x == -1 or y == -1:
            continue
        if x != y:
            return False
    return True

answers = []

def getFlag(cip, sboxes, mp):
    # get the flag based on current mapping, unknown char will be shown as '?'
    res = []
    for c in cip:
        afterMap = [mp.get(x, -1) for x in c]
        found = False
        for i, s in enumerate(sboxes):
            if s == afterMap:
                res.append(i)
                found = True
                break
        if not found:
            res.append(ord('?'))
    return bytes(res)

def brute(cip, sboxes, mp):
    """
    cip and sboxes remain unchanged throughout the recursive call,
    but I feel bad using global variables.
    """
    if DEBUG:
        print(getFlag(cip, sboxes, mp))

    # check is finished
    isFinished = True
    for c in cip:
        if all(x in mp for x in c):
            pass
        else:
            isFinished = False

    if isFinished:
        answers.append(getFlag(cip, sboxes, mp))
        print("Found an answer!!!!!!")
        return

    # try matching

```

```

isContradiction = False
mp = mp.copy()

# Find the one with least possible matches.
min_pos = 256
index = -1

for idx, c in enumerate(cip):
    afterMap = [mp.get(x, -1) for x in c]
    if -1 not in afterMap:
        continue

    matches = [s for s in sboxes if match(s, afterMap)]

    if len(matches) == 0:
        isContradiction = True
        break

    if min_pos > len(matches):
        index = idx
        min_pos = len(matches)

if isContradiction:
    return

# now bruteforce all possibilities
assert index != -1
afterMap = [mp.get(x, -1) for x in cip[index]]
matches = [s for s in sboxes if match(s, afterMap)]
for m in matches:
    for x, y in zip(cip[index], m):
        mp[x] = y
    brute(cip, sboxes, mp)

# This is based on the repetition
for _ in [132, 197]:
    mp = {35: 224, 109: 144, 4: _}
    brute(cip, sboxes, mp)

print("Answers:")
answers = list(set(answers))
for x in answers:
    print(b"sdctf{" + x + b"}")

# The fourth one is the actual answer

```

```

Ciphertext repetition:
[4, 5, 4, 6]
[34, 35, 36, 35]
[109, 60, 110, 109]
Sbox repetition:
[132, 93, 132, 211]
[197, 32, 197, 248]
[144, 86, 67, 144]
[165, 224, 27, 224]
Found an answer!!!!!!
Found an answer!!!!!!
Found an answer!!!!!!
Found an answer!!!!!!

```

```
Found an answer!!!!!!  
Found an answer!!!!!!  
Found an answer!!!!!!  
Found an answer!!!!!!  
Answers:  
b'sdctf{r0l1-LR~pWn.c6yPt0_wi7h,ECB:I5*b8d!KQvJmLxgX95saANMFSeU}'  
b'sdctf{r0l1-uR~pWn.c6yPt0_wi7h,ECB:I5*b8d!cQvJmLxgX9DsaANMFSeU}'  
b'sdctf{r0l1-uR~pWn.c6yPt0_wi7h,ECB:I5*b8d!KQvJmLxgX9DsaANMFSeU}'  
b'sdctf{r0l1-uR~pWn.c6yPt0_wi7h,ECB:I5*b8d!cQvJmLxgX95saANMFSeU}'  
b'sdctf{r0l1-LR~pWn.c6yPt0_wi7h,ECB:I5*b8d!KQvJmLxgX9DsaANMFSeU}'  
b'sdctf{r0l1-LR~pWn.c6yPt0_wi7h,ECB:I5*b8d!cQvJmLxgX9DsaANMFSeU}'  
b'sdctf{r0l1-uR~pWn.c6yPt0_wi7h,ECB:I5*b8d!KQvJmLxgX95saANMFSeU}'  
b'sdctf{r0l1-LR~pWn.c6yPt0_wi7h,ECB:I5*b8d!cQvJmLxgX95saANMFSeU}'
```

full script in [solve2.py](#)

Interactive controls are omitted from this PDF. See the canonical HTML version.